LonghornSilicon × Silicon-Agnostic Compiler
Integration Brief

Date: 2026-05-13 **Branch**: `compiler-integration-prep`
Contact: Chaithu Talasila **Email**: talasilachaithu105@gmail.com
Affil.: UT Austin **Repo**: LonghornSilicon/adaptive-precision-attention

1. What LonghornSilicon Is Building

A four-block LLM inference accelerator targeting TSMC 16FFC via the TSMC University Program (tape-out target Q3 / Q4 2026):

1. **ACU (Attention Compute Unit)** — INT8 / FP16 mixed-precision attention with a streaming *precision controller* that picks the precision per tile from a single-cycle ratio check.
   Status: RTL frozen, 253 / 253 tests pass, Sky130 PnR signed off (DRC / LVS / antenna / IR-drop clean), ASAP7 area projection, public paper.
2. **KV Cache Engine** — on-chip SRAM / eDRAM with compression (2–4 bit quantization plus outlier handling). *Not yet implemented.*
3. **Token Importance Unit** — per-token attention-weight accumulator driving keep / demote / evict for mixed-precision KV retention. *Not yet implemented.*
4. **Memory Hierarchy Controller** — L1 SRAM, L2 eDRAM, off-chip LPDDR5 with SHIELD-style refresh disable on short-lived data. *Not yet implemented.*

For this meeting, the precision controller (block 1) is what we can commit to a stable interface for.

2. What We Offer the Compiler Today

Three concrete artifacts the compiler team can target right now, before silicon or FPGA hardware exist:

Artifact	Purpose / Status
Bit-accurate Python reference model `sw/reference_model/precision_controller_ref.py`	A pure-Python class that produces exactly the INT8 / FP16 decision the chip will produce. Verified **143 / 143 bit-exact** against the RTL testbench.
ISA / interface specification `docs/isa/precision_controller_isa.pdf`	Versioned (`pc-isa-0.1`) memory map, AXI protocols, C API stub. Stable for the precision controller; will unify with other blocks as they land.
Compiler-perspective demo `sw/reference_model/example_compiler_use.py`	Three usage levels (naive / batched / calibrated) showing what compiler-emitted code looks like against the model.

A compiler engineer can write a backend against the Python model today and trust that the chip will agree byte-for-byte when silicon arrives.

3. Four-Phase Integration Plan

The interface is stable across all four phases. Same memory map, same streaming protocol, same model semantics throughout.

Phase	Timeline	Compiler targets	Who delivers what
0 (Python ref)	now	Python reference model	**Us:** model + ISA + the branch. **Compiler** → calls model
1 (FPGA prototype)	when ZCU102 / 104 arrives	AXI-Lite + AXI-Stream	**Us:** Vivado bitstream **Compiler team:** bac
2 (multi-block FPGA)	after KV cache + token + memory blocks complete	Same AXI, more blocks visible	**Us:** larger Vivado pro **team:** extend backend
3 (silicon)	2027+ (post-tape-out)	PCIe-attached chip	**Us:** chip + Linux dri **team:** re-target backe

Critically: phase 0 is enough to start. The compiler team does not need to wait for anything from us beyond what is already on the branch.

4. What We Ask of the Compiler Team

1. A short architecture writeup: what is the IR (MLIR / Relay / custom), what is the typical hardware-target shape they integrate against, what experience do they have adding new backends.
2. A scoping conversation: is the precision controller the right first target, or do they want to wait for more blocks?
3. Honest timeline expectations: phase 0 work could start immediately; phase 1 (FPGA) needs the ZCU102 / 104 in hand, which is a separate gating item.
4. Their thoughts on the open ISA questions (§5): runtime-vs-synthesis-time `THRESHOLD`, exposing `max` / `sum` alongside the decision bit, AXI vs other fabric.

5. Open Questions for the Meeting

These are intentionally on the table; we have not committed to them.

1. Runtime-configurable `THRESHOLD`? Cost: one register, one parameterized multiplier. Benefit: per-workload tuning without re-synthesis.
2. Single-bit decision vs. exposing `max` and `sum`? Lets the compiler implement its own gating policy downstream.
3. AXI-Lite + AXI-Stream the right fabric, or CHI / TileLink / custom?
4. Phase 1 timing: do they need real FPGA hardware to begin meaningful work, or is the phase 0 Python target enough to start a backend that the FPGA later validates?

6. What We Are *Not* Committing To Today

To keep boundaries clear up front:

- Schedule promises tied to tape-out — the TSMC University Program shuttle date is outside our control.
- An ISA for the full chip — three of the four blocks are not yet RTL. The precision-controller ISA is stable; the chip-level ISA is not.
- Source access to private repos — until we know the collaboration shape, the public repo is the boundary.
- Exclusivity — other compiler projects may also be interested in this chip. Happy to discuss exclusivity terms, but not by default.

7. Concrete Next Step (Roughly One Week)

If the meeting goes well, the smallest meaningful collaboration looks like:

1. The compiler team writes a two-page integration plan: which IR ops map to which ISA operations, what their backend's runtime expects from us, what would unblock them to start coding.
2. We respond within a week with the gaps we can fill and timelines.
3. They pick the first integration target — likely a single matmul or a single attention block exercised against the phase 0 reference model.

4. We co-author the first commit in their backend that produces correct decisions when run against our model.

That commit becomes the milestone we can both demo at meetings, conferences, or to UT Austin / TSMC for further support.

Everything in this brief, plus the source code, RTL, OpenLane-signed-off Sky130 GDS, and the precision-controller paper, is public and shareable on the `compiler-integration-prep` *branch of* `LonghornSilicon/adaptive-precision-attentio`
The TSMC 16FFC PDK is NDA-protected and not part of this material.